

TUGAS
SISTEM OPERASI



Disusun oleh :
Muhammad Rafi Hendardi
2410651062

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNIK
UNIVERSITAS MUHAMMADIYAH JEMBER
2025

Program

Deadlock :

```
#include <stdio.h>

#include <stdlib.h>

#include <pthread.h>

#include <semaphore.h>

#include <unistd.h> // Untuk sleep() dan usleep()


#define N 5 // Jumlah Filsuf


sem_t chopsticks[N];


void *philosopher(void *num) {
    int i = *(int *)num;
    int left = i;
    int right = (i + 1) % N;


    while(1) {
        printf("Filsuf %d sedang berpikir...\n", i);
        usleep(rand() % 1000000 + 500000); // Berpikir (0.5 - 1.5 detik)


        printf("Filsuf %d lapar dan mencoba mengambil sumpit kiri %d\n", i, left);


        sem_wait(&chopsticks[left]);
        printf("Filsuf %d mengambil sumpit kiri %d\n", i, left);
```

```
usleep(500000);
```

```
printf("Filsuf %d mencoba mengambil sumpit kanan %d\n", i, right);
```

FILSUF

```
sem_wait(&chopsticks[right]);
```

```
printf("Filsuf %d sedang makan!\n", i);
```

```
usleep(2000000);
```

```
sem_post(&chopsticks[right]);
```

```
sem_post(&chopsticks[left]);
```

```
printf("Filsuf %d selesai makan dan meletakkan sumpit.\n", i);
```

```
}
```

```
return NULL;
```

```
}
```

```
int main() {
```

```
pthread_t p_threads[N];
```

```
int p_ids[N];
```

```
int i;
```

```
srand(time(NULL));
```

```
printf("=== DEADLOCK DEMONSTRATION (Ambil Kiri Lalu Kanan) ===\n");
```

```

for (i = 0; i < N; i++) {
    sem_init(&chopsticks[i], 0, 1);
}

for (i = 0; i < N; i++) {
    p_ids[i] = i;
    pthread_create(&p_threads[i], NULL, philosopher, &p_ids[i]);
}

for (i = 0; i < N; i++) {
    pthread_join(p_threads[i], NULL);
}

for (i = 0; i < N; i++) {
    sem_destroy(&chopsticks[i]);
}

printf("Program selesai.\n");

return 0;
}

```

Solusi A

```

#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>

```

```

#include <unistd.h>

#include <time.h>


#define N 5 // Jumlah Filsuf
#define MAX_TIME 60 // Durasi berjalan (60 detik)


// --- Variabel Global ---

sem_t chopsticks[N];
int eat_count[N];
time_t start_time;


// Fungsi untuk mendapatkan timestamp [MM:SS]
long get_elapsed_seconds() {
    return time(NULL) - start_time;
}


// Fungsi untuk logging dengan format dan emoji
void log_action(int p_id, const char *msg, const char *emoji) {
    long elapsed = get_elapsed_seconds();
    printf("[%0.2ld:%0s%0.2ld] %s Filsuf-%d %s\n",
        elapsed / 60, (elapsed < 10) ? "0" : "", elapsed % 60,
        emoji, p_id + 1, msg);
    fflush(stdout);
}


// -----
// --- FUNGSI FILSUF (Resource Ordering) ---

```

```
// -----
```

```
void *philosopher(void *num) {
```

```
    int i = *(int *)num;
```

```
    int left_c = i;
```

```
    int right_c = (i + 1) % N;
```

```
    int first, second;
```

```
    // Resource Ordering Logic: Filsuf Genap ambil Kiri, Ganjil ambil Kanan Dulu
```

```
    if (i % 2 == 0) { // Filsuf Genap (0, 2, 4)
```

```
        first = left_c;
```

```
        second = right_c;
```

```
    } else { // Filsuf Ganjil (1, 3)
```

```
        first = right_c;
```

```
        second = left_c;
```

```
    }
```

```
    if (first == second) {
```

```
        first = left_c;
```

```
        second = right_c;
```

```
    }
```

```
    while (get_elapsed_seconds() < MAX_TIME) {
```

```
        log_action(i, "sedang berpikir...", "🤔");
```

```
        usleep((rand() % 1000 + 500) * 1000); // 0.5-1.5s thinking
```

```

log_action(i, "lapar, mencoba ambil sumpit...", "🍴");

// 1. Ambil Sumpit PERTAMA (Resource Kecil)
log_action(i, "mengambil sumpit pertama...", "🍴");
sem_wait(&chopsticks[first]);

// 2. Ambil Sumpit KEDUA (Resource Besar)
log_action(i, "mengambil sumpit kedua...", "🍴");
sem_wait(&chopsticks[second]);

// Sukses Makan
log_action(i, "sedang MAKAN...", "😋");
eat_count[i]++;
usleep((rand() % 1000 + 500) * 1000); // 0.5-1.5s eating

// Lepaskan sumpit
sem_post(&chopsticks[second]);
sem_post(&chopsticks[first]);
log_action(i, "menaruh kedua sumpit", "🍴");
}
return NULL;
}

// -----
// --- FUNGSI UTAMA (Resource Ordering) ---
// -----

```

```

int main() {
    pthread_t p_threads[N];
    int p_ids[N];
    int i;

    srand(time(NULL));
    for(i=0; i<N; i++) eat_count[i] = 0;
    start_time = time(NULL);

    printf("\n\n=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Solusi: Resource Ordering (A)\n");

    // Inisialisasi Semaphores (Sumpit)
    for (i = 0; i < N; i++) {
        sem_init(&chopsticks[i], 0, 1);
    }

    // Buat Threads
    for (i = 0; i < N; i++) {
        p_ids[i] = i;
        pthread_create(&p_threads[i], NULL, philosopher, &p_ids[i]);
    }

    // Tunggu Threads Selesai
    for (i = 0; i < N; i++) {
        pthread_join(p_threads[i], NULL);
    }
}

```



```
}
```

```
// Hapus Semaphores
```

```
for (i = 0; i < N; i++) {  
    sem_destroy(&chopsticks[i]);  
}
```

```
// Tampilkan Statistik
```

```
printf("\n=== STATISTIK (%d detik) ===\n", MAX_TIME);  
int total_eats = 0;  
int min_eats = 99999, max_eats = 0;  
for (i = 0; i < N; i++) {  
    printf("Filsuf-%d: Makan %d kali\n", i + 1, eat_count[i]);  
    total_eats += eat_count[i];  
    if (eat_count[i] < min_eats) min_eats = eat_count[i];  
    if (eat_count[i] > max_eats) max_eats = eat_count[i];  
}
```

```
printf("\n✅ Total berhasil makan: %d kali\n", total_eats);  
printf("✅ Tidak ada deadlock! (Program berjalan hingga selesai)\n");
```

```
if (max_eats - min_eats <= 2) {  
    printf("✅ Distribusi fairness: BAIK\n");  
} else {  
    printf("✅ Distribusi fairness: CUKUP/KURANG (Rentang %d)\n", max_eats - min_eats);  
}
```

```
    return 0;
}
```

Solusi B

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#define N 5 // Jumlah Filsuf
```

```
#define MAX_TIME 60 // Durasi berjalan (60 detik)
```

```
// --- Variabel Global ---
```

```
sem_t chopsticks[N];
```

```
sem_t room; // Monitor/Room Semaphore (Limit N-1)
```

```
int eat_count[N];
```

```
time_t start_time;
```

```
// Fungsi untuk mendapatkan timestamp [MM:SS]
```

```
long get_elapsed_seconds() {
```

```
    return time(NULL) - start_time;
```

```
}
```

```
// Fungsi untuk logging dengan format dan emoji
```

```
void log_action(int p_id, const char *msg, const char *emoji) {
```

```
    long elapsed = get_elapsed_seconds();
```

```

printf("[%0.2ld:%s%.2ld] %s Filsuf-%d %s\n",
        elapsed / 60, (elapsed < 10) ? "0" : "", elapsed % 60,
        emoji, p_id + 1, msg);
fflush(stdout);
}

// -----
// --- FUNGSI FILSUF (Limiting Philosophers) ---
// -----

void *philosopher(void *num) {
    int i = *(int *)num;
    int left_c = i;
    int right_c = (i + 1) % N;

    while (get_elapsed_seconds() < MAX_TIME) {
        log_action(i, "sedang berpikir...", "🧠");
        usleep((rand() % 1000 + 500) * 1000); // 0.5-1.5s thinking

        log_action(i, "lapar, mencoba masuk ruangan...", "🍲");
        // 0. Coba masuk Ruangan (Limit 4 Filsuf)
        sem_wait(&room);

        log_action(i, "berhasil masuk ruangan...", "🍲");

        // 1. Ambil Sumpit KIRI
        log_action(i, "mengambil sumpit kiri...", "🍴");
    }
}

```

```

sem_wait(&chopsticks[left_c]);

// 2. Ambil Sumpit KANAN
log_action(i, "mengambil sumpit kanan...", "🥢");
sem_wait(&chopsticks[right_c]);

// Sukses Makan
log_action(i, "sedang MAKAN...", "😋");
eat_count[i]++;
usleep((rand() % 1000 + 500) * 1000); // 0.5-1.5s eating

// Lepaskan sumpit
sem_post(&chopsticks[right_c]);
sem_post(&chopsticks[left_c]);
log_action(i, "menaruh kedua sumpit", "🥢");

// 3. Keluar Ruangan
sem_post(&room);
log_action(i, "keluar ruangan", "🧑");
}
return NULL;
}

// -----
// --- FUNGSI UTAMA (Limiting Philosophers) ---
// -----

```

```

int main() {
    pthread_t p_threads[N];
    int p_ids[N];
    int i;

    srand(time(NULL));
    for(i=0; i<N; i++) eat_count[i] = 0;
    start_time = time(NULL);

    printf("\n\n=== DINING PHILOSOPHERS PROBLEM ===\n");
    printf("Solusi: Limiting Philosophers (B)\n");

    // Inisialisasi Semaphores
    for (i = 0; i < N; i++) {
        sem_init(&chopsticks[i], 0, 1);
    }
    sem_init(&room, 0, N - 1); // Solusi B: Monitor/Room Semaphore

    // Buat Threads
    for (i = 0; i < N; i++) {
        p_ids[i] = i;
        pthread_create(&p_threads[i], NULL, philosopher, &p_ids[i]);
    }

    // Tunggu Threads Selesai
    for (i = 0; i < N; i++) {
        pthread_join(p_threads[i], NULL);
    }
}

```

```
}
```

```
// Hapus Semaphores
```

```
for (i = 0; i < N; i++) {  
    sem_destroy(&chopsticks[i]);  
}
```

```
sem_destroy(&room);
```

```
// Tampilkan Statistik
```

```
printf("\n=== STATISTIK (%d detik) ===\n", MAX_TIME);
```

```
int total_eats = 0;
```

```
int min_eats = 99999, max_eats = 0;
```

```
for (i = 0; i < N; i++) {  
    printf("Filsuf-%d: Makan %d kali\n", i + 1, eat_count[i]);  
    total_eats += eat_count[i];  
    if (eat_count[i] < min_eats) min_eats = eat_count[i];  
    if (eat_count[i] > max_eats) max_eats = eat_count[i];  
}
```

```
printf("\n✅ Total berhasil makan: %d kali\n", total_eats);
```

```
printf("✅ Tidak ada deadlock! (Program berjalan hingga selesai)\n");
```

```
if (max_eats - min_eats <= 2) {
```

```
    printf("✅ Distribusi fairness: BAIK\n");
```

```
} else {
```

```
    printf("✅ Distribusi fairness: CUKUP/KURANG (Rentang %d)\n", max_eats - min_eats);
```

```
}
```

```
    return 0;  
}
```

Outputnya

```

rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ nano deadlock_demonstration.c
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ gcc deadlock_demonstration.c -o deadlock_demonstration
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_demonstration$ ./deadlock_demonstration
=== DEMONSTRASI DEADLOCK (Dining Philosophers) ===
Filsuf 0 sedang berpikir...
Filsuf 1 sedang berpikir...
Filsuf 2 sedang berpikir...
Filsuf 3 sedang berpikir...
Filsuf 4 sedang berpikir...
Filsuf 2 mencoba mengambil sumpit kiri 2
Filsuf 2 berhasil mengambil sumpit kiri 2
Filsuf 1 mencoba mengambil sumpit kiri 1
Filsuf 1 berhasil mengambil sumpit kiri 1
Filsuf 3 mencoba mengambil sumpit kiri 3
Filsuf 4 mencoba mengambil sumpit kiri 4
Filsuf 4 berhasil mengambil sumpit kiri 4
Filsuf 3 berhasil mengambil sumpit kiri 3
Filsuf 0 mencoba mengambil sumpit kiri 0
Filsuf 0 berhasil mengambil sumpit kiri 0
Filsuf 2 mencoba mengambil sumpit kanan 3
Filsuf 1 mencoba mengambil sumpit kanan 2
Filsuf 4 mencoba mengambil sumpit kanan 0
Filsuf 3 mencoba mengambil sumpit kanan 4
Filsuf 0 mencoba mengambil sumpit kanan 1

```

```

[00:59] [0] Filsuf-5 sedang berpikir...
[00:59] [0] Filsuf-4 mengambil sumpit kedua...
[00:59] [0] Filsuf-4 sedang MAKAN...
[01:00] [00] Filsuf-2 menaruh kedua sumpit
[01:00] [0] Filsuf-3 mengambil sumpit kedua...
[01:00] [0] Filsuf-1 lapar, mencoba ambil sumpit...
[01:00] [0] Filsuf-1 mengambil sumpit pertama...
[01:00] [0] Filsuf-1 mengambil sumpit kedua...
[01:00] [0] Filsuf-1 sedang MAKAN...
[01:00] [00] Filsuf-4 menaruh kedua sumpit
[01:00] [0] Filsuf-3 sedang MAKAN...
[01:01] [0] Filsuf-5 lapar, mencoba ambil sumpit...
[01:01] [0] Filsuf-5 mengambil sumpit pertama...
[01:01] [0] Filsuf-5 mengambil sumpit kedua...
[01:01] [00] Filsuf-1 menaruh kedua sumpit
[01:01] [0] Filsuf-5 sedang MAKAN...
[01:02] [00] Filsuf-5 menaruh kedua sumpit
[01:02] [00] Filsuf-3 menaruh kedua sumpit

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 25 kali
Filsuf-2: Makan 23 kali
Filsuf-3: Makan 22 kali
Filsuf-4: Makan 21 kali
Filsuf-5: Makan 22 kali

[00] Total berhasil makan: 113 kali
[00] Tidak ada deadlock! (Program berjalan hingga selesai)
[00] Distribusi fairness: CUKUP/KURANG (Rentang 4)
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_prevention$

```



```
rafi@DESKTOP-TM81UKJ: ~/Tugas5/deadlock_prevention
[01:00] [0] Filsuf-5 menaruh kedua sumpit
[01:00] [0] Filsuf-5 keluar ruangan
[01:00] [0] Filsuf-4 sedang MAKAN...
[01:00] [0] Filsuf-1 lapar, mencoba masuk ruangan...
[01:00] [0] Filsuf-1 berhasil masuk ruangan...
[01:00] [0] Filsuf-1 mengambil sumpit kiri...
[01:00] [0] Filsuf-1 mengambil sumpit kanan...
[01:00] [0] Filsuf-2 menaruh kedua sumpit
[01:00] [0] Filsuf-2 keluar ruangan
[01:00] [0] Filsuf-3 mengambil sumpit kanan...
[01:00] [0] Filsuf-1 sedang MAKAN...
[01:01] [0] Filsuf-4 menaruh kedua sumpit
[01:01] [0] Filsuf-4 keluar ruangan
[01:01] [0] Filsuf-3 sedang MAKAN...
[01:01] [0] Filsuf-1 menaruh kedua sumpit
[01:01] [0] Filsuf-1 keluar ruangan
[01:02] [0] Filsuf-3 menaruh kedua sumpit
[01:02] [0] Filsuf-3 keluar ruangan

=== STATISTIK (60 detik) ===
Filsuf-1: Makan 23 kali
Filsuf-2: Makan 23 kali
Filsuf-3: Makan 24 kali
Filsuf-4: Makan 22 kali
Filsuf-5: Makan 23 kali

[0] Total berhasil makan: 115 kali
[0] Tidak ada deadlock! (Program berjalan hingga selesai)
[0] Distribusi fairness: BAIK
rafi@DESKTOP-TM81UKJ:~/Tugas5/deadlock_prevention$
```

ANALISI MENDALAM

Mengapa Deadlock Bisa Terjadi?

Deadlock terjadi ketika empat kondisi Coffman terpenuhi secara simultan. Dalam konteks masalah *Dining Philosophers*, deadlock yang terjadi dalam program **Demonstration** disebabkan oleh:

1. **Mutual Exclusion:** Sumpit adalah sumber daya eksklusif (diwakili oleh `sem_wait`).
2. **Hold and Wait:** Setiap filsuf memegang sumpit kirinya (`sem_wait(&chopsticks[left])`) sambil menunggu sumpit kanannya.
3. **No Preemption:** Sumpit tidak dapat direbut paksa.
4. **Circular Wait:** Semua filsuf membentuk rantai tunggu melingkar, di mana P_i menunggu sumpit yang dipegang oleh $P_{(i+1)\%N}$.

Bagaimana Masing-Masing Solusi Bekerja?

Solusi A: Resource Ordering

- **Cara Kerja:** Solusi ini menghilangkan kondisi **Circular Wait**. Dengan menetapkan aturan bahwa semua filsuf (kecuali satu) harus mengambil sumpit dalam urutan nomor sumber daya (kecil > besar), dan satu filsuf mengambil dalam urutan terbalik (besar > kecil), rantai tunggu melingkar mustahil terbentuk.

- **Inti:** Jika Filsuf 4 mengambil sumpit 0 dulu (besar) dan menunggu sumpit 4 (kecil), sementara Filsuf 3 memegang sumpit 3 (kecil) dan menunggu sumpit 4 (besar), maka Filsuf 4 tidak akan pernah menunggu Filsuf 3, dan rantai terputus.

Solusi B: Limiting Philosophers

- **Cara Kerja:** Solusi ini menggunakan semaphore tambahan (room) dengan nilai N-1 (yaitu 4) untuk menghilangkan kondisi **Hold and Wait** pada tingkat sistem. Dengan membatasi hanya 4 filsuf yang dapat masuk ke "ruangan" untuk mengambil sumpit, selalu ada setidaknya satu sumpit yang bebas yang dapat diambil oleh salah satu dari 4 filsuf tersebut.
- **Inti:** Selalu ada satu pasang sumpit yang bisa diamankan sepenuhnya oleh salah satu filsuf di ruangan, yang kemudian melepaskan sumpitnya dan membuka jalan bagi filsuf lain.

Solusi Mana yang Paling Adil (Fair)?

Solusi B (Limiting Philosophers) cenderung paling adil (fair).

- **Alasan:** Solusi B menggunakan satu *semaphore* sentral (room) sebagai *gatekeeper*. Ketika seorang filsuf melepaskan sumpit dan keluar dari ruangan (`sem_post(&room)`), filsuf berikutnya yang lapar memiliki peluang yang kurang lebih sama untuk masuk. Ini membantu mendistribusikan waktu makan lebih merata.
- **Perbandingan dengan Solusi A:** Meskipun Solusi A efisien, filsuf yang urutan pengambilannya dibalik (misalnya Filsuf 4) secara teori mungkin memiliki sedikit kerugian dibandingkan tetangganya karena harus menunggu sumpit dengan nomor besar yang juga menjadi sumpit kecil bagi tetangga di sisi kirinya.

Apakah Ada Kemungkinan Starvation (Kelaparan)?

Ya, meskipun *deadlock* dicegah, *starvation* (kelaparan) tetap mungkin terjadi pada kedua solusi, terutama pada implementasi ini yang menggunakan `usleep()` acak.

- ***Starvation* di Solusi A:** Filsuf yang memiliki urutan pengambilan yang kurang menguntungkan (misalnya, yang harus menunggu sumpit nomor besar) dapat mengalami *starvation* jika tetangganya secara konsisten dan cepat makan dan kembali ke kondisi lapar.
- ***Starvation* di Solusi B:** Meskipun *fairness*-nya baik, *starvation* masih mungkin terjadi pada tingkat *semaphore*. Jika banyak filsuf terus menerus kembali ke kondisi lapar dan bersaing di pintu masuk ruangan (`sem_wait(&room)`), filsuf yang terjebak di antrian semaphore dapat terus menerus diabaikan (*thread scheduler* atau alokasi sumber daya bisa menjadi bias).

